



Spring Data JPA: Query Methods, Paging, and Sorting

Section 1: Learn

What Is Spring Data JPA?

Spring Data JPA is a powerful abstraction over the JPA (Java Persistence API) that allows you to interact with relational databases using repository interfaces. It simplifies common data access patterns like queries, pagination, and sorting.

Why Use It?

- **No SQL Required:** Auto-generates queries based on method names.
- **Fast Development:** Reduces boilerplate for CRUD and complex queries.
- **Built-in Pagination and Sorting:** Handle large datasets efficiently.

Query Method Naming Pattern

1. **Prefix:** **find**, **get**, **read**, **count**, **exists**
2. **By:** Starts the condition
3. **Property:** Entity field name
4. **Operators:** **And**, **Or**, **Between**, **Like**, **LessThan**

Interesting Anecdote

Spring Data JPA lets developers avoid 80% of standard SQL queries simply by naming methods correctly—drastically reducing time spent writing, testing, and debugging SQL.



Section 2: Practice

Example Query Methods

```
List<User> findByUsername(String username);  
List<User> findByEmailAndActive(String email, boolean active);  
List<User> findByAgeLessThan(int age);  
List<User> findByCreatedDateBetween(Date start, Date end);
```

Pattern Matching and Boolean

```
List<User> findByNameLike(String pattern);  
List<User> findByActiveTrue();  
List<User> findByDeletedIsNull();
```

Repository Interface Example

```
public interface UserRepository extends JpaRepository<User, Long> {  
    List<User> findByEmail(String email);  
    User findByUsername(String username);  
    List<User> findByAgeBetween(int start, int end);  
}
```

Using @Query Annotation

```
@Query("SELECT p FROM Product p WHERE p.price > ?1")  
List<Product> findExpensiveProducts(BigDecimal price);  
  
@Query(value = "SELECT * FROM products WHERE category = :cat",  
nativeQuery = true)
```



```
List<Product> findByCategoryNative(@Param("cat") String category);
```

Custom Repository Implementation

```
public interface UserRepositoryCustom {  
    List<User> findActiveUsersByRole(String role);  
}  
  
public class UserRepositoryImpl implements UserRepositoryCustom {  
    @PersistenceContext  
    private EntityManager entityManager;  
  
    public List<User> findActiveUsersByRole(String role) {  
        // Use JPQL or Criteria API here  
    }  
}  
  
public interface UserRepository extends JpaRepository<User, Long>,  
    UserRepositoryCustom {}
```

Pagination with Pageable

```
Page<User> findAll(Pageable pageable);  
Page<User> findByCategory(String category, Pageable pageable);  
Slice<User> findByStatus(String status, Pageable pageable);
```



Create Pageable Object

```
Sort sort =  
Sort.by("lastName").ascending().and(Sort.by("firstName").ascending());  
Pageable pageable = PageRequest.of(0, 20, sort);  
Page<User> page = userRepository.findAll(pageable);
```

Working with Page Results

```
int currentPage = page.getNumber();  
int pageSize = page.getSize();  
long totalElements = page.getTotalElements();  
int totalPages = page.getTotalPages();  
List<User> users = page.getContent();  
boolean hasNext = page.hasNext();
```

Section 3: Know More

FAQs

Q1: What happens if I write a query method that doesn't match a field name?

- **Answer:** Spring will throw an exception indicating the field doesn't exist.

Q2: Can I mix @Query and derived methods?

- **Answer:** Yes, you can use both in the same repository.

Q3: What is the difference between Page and Slice?

- **Answer:** **Page** includes metadata (total pages/elements), while **Slice** does not—better for large datasets.

Q4: Can pagination and sorting be used together?



- **Answer:** Yes, just pass a **PageRequest** object with sorting instructions.

Q5: When should I use custom implementations?

- **Answer:** For queries that are too complex for method names or when using dynamic conditions with EntityManager.

Spring Data JPA offers a declarative and flexible way to perform database operations. Mastering query methods and pagination boosts productivity and keeps your code clean.